

Handwriting Recognition in SQL

Groner's GRAIL Recognizer as a Relational Pipeline

Micha Riedlinger

June 16, 2026

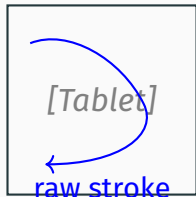
University of Tübingen

Seminar: *SQL is a Programming Language*

Problem

What is Handwriting Recognition?

- Input: sequence of (x, y) coordinates captured by a stylus
- Goal: identify the drawn character (letter or digit)
- Origin: GRAIL system (RAND Corporation, 1966) — real-time recognition on an IBM System/360



Why Is This Interesting?

The key challenge

- tablet discretisation introduces jaggies in every stroke
- A single character produces dozens of noisy, redundant coordinate points
- two people writing the same letter look very different on paper
- two different characters can have many similarities
- ideally we can recognize the character fast

Example (the digit 5 vs the letter S)

- Drawn in one stroke, left → down → right → down → left
- Corners resolve the ambiguity

Solution: The Pipeline

Approache to Character Recognition

Feature-based (Groner)

- Extract compact stroke features
- Rule-based decision tree
- Fast, interpretable, works in real-time

Why Groner's pipeline suits SQL

Each processing stage is a *set transformation* on an ordered sequence of points — a natural fit for relational queries with window functions and recursive CTEs.

Groner's Five-Stage Pipeline



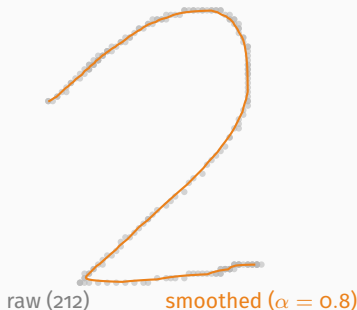
Hier ggf. an jeden Pipeline Schritt notieren, welche Herausforderung gefixt wird.

- Each stage is one SQL WITH clause — the full pipeline is a single query
- Running example: digit **3** passes through each stage

Stage 1 — Smoothing

Remove quantisation noise with an moving average: each smoothed point is a weighted blend of the previous smoothed point and the new raw point.

$$x_i^s = \alpha \cdot x_{i-1}^s + (1 - \alpha) \cdot x_i^r \quad y_i^s = \alpha \cdot y_{i-1}^s + (1 - \alpha) \cdot y_i^r$$



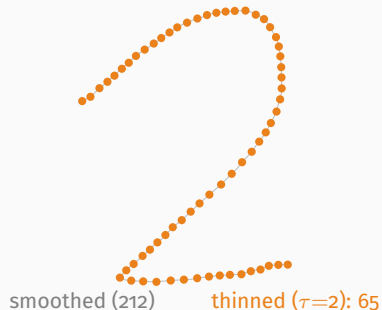
Stage 1 — Smoothing Implementation

```
1 SELECT pos,  
2     x,  
3     y  
4 FROM raw_data  
5 WHERE pos = 1  
6 UNION ALL  
7 SELECT r.pos,  
8     get('alpha')*s.x + (1 - get('alpha')) * r.x AS sx,  
9     get('alpha')*s.y + (1 - get('alpha')) * r.y AS sy  
10 FROM raw_data r  
11     JOIN smoothing_phase s ON r.pos = s.pos + 1
```

Stage 2 — Thinning

Discard points too close to the last accepted point (x_{j-1}^t, y_{j-1}^t) to remove redundancy from slow pen movement:

$$\text{accept } (x_i^s, y_i^s) \iff \sqrt{(x_i^s - x_{j-1}^t)^2 + (y_i^s - y_{j-1}^t)^2} > \tau$$



- Threshold τ defines an acceptance radius around the last kept point
- Original GRAIL used a bounding-square test ($|\Delta x| \geq \tau$ or $|\Delta y| \geq \tau$); we use Euclidean distance — rotation-invariant
- Pipeline value: $\tau = 2$ (raw pixel scale)

Stage 2 — Thinning Implementation

```
1  SELECT pos, x, y
2  FROM smoothing_phase
3  WHERE pos = 1
4  UNION ALL
5  SELECT next_keep.pos, next_keep.x, next_keep.y
6  FROM thinning_scan ts
7  CROSS JOIN LATERAL (
8      SELECT pos, x, y
9      FROM smoothing_phase
10     WHERE pos > ts.pos
11           AND sqrt((x - ts.x)^2+(y - ts.y)^2) > getvariable('tau')
12     ORDER BY pos
13     LIMIT 1
14 ) next_keep
```

Stage 3 — Curvature Detection

Assign one of four cardinal directions to each segment by comparing the bounding rectangle's width to its height ($\Delta x = x_i^t - x_{i-1}^t$, $\Delta y = y_i^t - y_{i-1}^t$):

$$d_i = \begin{cases} \text{right/left} & \text{if } |\Delta x| \geq |\Delta y|, \text{ sign of } \Delta x \\ \text{up/down} & \text{if } |\Delta x| < |\Delta y|, \text{ sign of } \Delta y \end{cases}$$

Only *changes* in direction are kept — e.g. **L** becomes $\downarrow \rightarrow$, **O** becomes $\leftarrow \downarrow \rightarrow \uparrow$.

Here a example + visualization like in the blog article!

Stage 3 — Curvature Detection Implementation

```
1 SELECT pos, x, y,  
2     CASE  
3         WHEN LAG(x) OVER w IS NULL THEN NULL  
4         WHEN ABS(x - LAG(x) OVER w) >= ABS(y - LAG(y) OVER w)  
5         THEN CASE WHEN x >= LAG(x) OVER w THEN 'right' ELSE 'left'  
6                 END  
7         ELSE CASE WHEN y >= LAG(y) OVER w THEN 'up' ELSE 'down'  
8                 END  
9     END AS direction  
10 FROM thinning_scan  
11 WINDOW w AS (ORDER BY pos)
```

Stage 4 — Corner Detection

Corners need finer angular resolution — encode each segment as one of 16 sectors via $\theta_i = \text{atan2}(\Delta y, \Delta x)$:

$$d_i = \left\lfloor \frac{(\theta_i + 11.25^\circ) \bmod 360^\circ}{22.5^\circ} \right\rfloor \in \{0, \dots, 15\}$$

A corner is flagged at point i when all three hold:

- Two segments before share a direction: $d_{i-1} = d_{i-2}$
- Two segments after share a direction: $d_{i+1} = d_{i+2}$
- Angular change $\geq 90^\circ$ (4 of 16 sectors), with wrap-around:

$$\min(|d_{i+1} - d_{i-1}|, 16 - |d_{i+1} - d_{i-1}|) \geq 4$$

Here a example + visualization like in the blog article!

Stage 4 — Corner Detection Implementation

```
1 SELECT pos, x, y
2 FROM directions_16_calc
3 WHERE direction IS NOT NULL
4 WINDOW w3 AS (ORDER BY pos)
5 QUALIFY
6     LEAST(ABS(LEAD(direction, 1) OVER w3 - LAG(direction, 1) OVER
7         w3),
8         16 - ABS(LEAD(direction, 1) OVER w3 - LAG(direction, 1)
9             OVER w3)) >= 4
10 AND LAG(direction, 1) OVER w3 = LAG(direction, 2) OVER w3
11 AND LEAD(direction, 1) OVER w3 = LEAD(direction, 2) OVER w3
```

Stage 5 — Character Recognition

Combine direction sequence, corner count/position, start/end cell, and aspect ratio to identify the character:

```
1 character_recognition AS (  
2     SELECT DISTINCT cdr.char  
3     FROM first_directions fd  
4     CROSS JOIN character_direction_rules cdr  
5     WHERE  
6         fd.dirs[1:len(cdr.prefix)] = cdr.prefix  
7 )
```

Noch nicht vollständig!

Putting It All Together

The complete recogniser is *one SQL query*:

```
1 WITH
2   smoothed  AS ( ... ),    -- Stage 1: window avg
3   thinned   AS ( ... ),    -- Stage 2: recursive CTE
4   directions AS ( ... ),    -- Stage 3: CASE + LAG
5   corners   AS ( ... ),    -- Stage 4: direction diff
6   result    AS ( ... )     -- Stage 5: CASE cascade
7 SELECT character FROM result;
```

Demo

Hier kurze Beschreibung für Setup, damit jedy es im Nachhinein einfach selbst testen kann.

Conclusion

Limitations & Outlook

Limitations

- Single-stroke characters only (Groner's original: multi-stroke)
- Decision tree hand-coded — biased toward one writing style
- Recursive thinning: row-by-row, not easily parallelised

Outlook

- Extend to multi-stroke via pen-up/pen-down events
- Learn thresholds from data (SQL + ML hybrid)
- Compare with modern DNN approaches (trade-offs)

Summary

- **Problem:** real-time recognition of handprinted characters from noisy coordinate streams
- **Solution:** Groner's 5-stage feature-extraction pipeline, implemented as a single SQL WITH query
- **Key SQL constructs:** window functions (smoothing, direction), recursive CTE (thinning), CASE cascade (decision tree)

References i